

Chapter 22: Discrete Optimization

Algorithms for Optimization, 2nd ed. (2026)

Kochenderfer & Wheeler

Lecture Notes

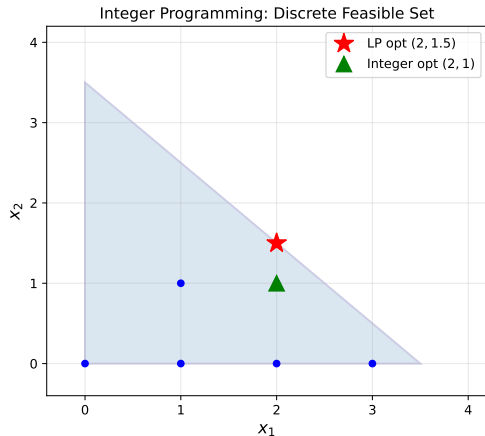
2026

Chapter Outline

- 1 Introduction
- 2 Integer Programs
- 3 Rounding (LP Relaxation)
- 4 Cutting Planes (Gomory Cuts)
- 5 Branch and Bound
- 6 Dynamic Programming
- 7 Ant Colony Optimization
- 8 Summary & Exercises

What is Discrete Optimization?

- Previous chapters: design variables are **continuous**
- Many real problems have **naturally discrete** variables:
 - Manufacturing: number of components
 - Scheduling: job assignment
 - Network routing: path selection
- Design variables chosen from a **discrete (finite) set**
- Cannot generally use gradient-based methods
- Methods covered: rounding, cutting planes, branch & bound, dynamic programming, ant colony optimization



Discrete feasible points (blue dots) vs. LP relaxation (red star) and integer optimum (green triangle).

Integer Programs — Standard Form

Definition (Integer Program)

An **integer program (IP)** is a linear program with integral constraints on design variables. Standard (inequality) form:

$$\begin{aligned} & \underset{\mathbf{x}}{\text{minimize}} && \mathbf{c}^\top \mathbf{x} \\ & \text{subject to} && \mathbf{Ax} \leq \mathbf{b} \\ & && \mathbf{x} \geq \mathbf{0} \\ & && \mathbf{x} \in \mathbb{Z}^n \end{aligned} \tag{22.1}$$

where $\mathbb{Z}^n$ is the set of n -dimensional integral vectors, $\mathbf{c} \in \mathbb{R}^n$, $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$.

- Also called *integer linear programs* (ILPs)
- Modern solvers (Gurobi, CPLEX) handle millions of variables

Equality Form and Mixed-Integer Programs

Equality Form (22.2)

$$\begin{array}{ll}\min_{\mathbf{x}, \mathbf{s}} & \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} & \mathbf{Ax} + \mathbf{s} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0}, \mathbf{s} \geq \mathbf{0} \\ & \mathbf{x} \in \mathbb{Z}^n\end{array}$$

Slack variables $\mathbf{s}$ need *not* be integral.

Mixed-Integer Program (22.3)

$$\begin{array}{ll}\min_{\mathbf{x}} & \mathbf{c}^\top \mathbf{x} \\ \text{s.t.} & \mathbf{Ax} = \mathbf{b} \\ & \mathbf{x} \geq \mathbf{0} \\ & \mathbf{x}_{\mathcal{D}} \in \mathbb{Z}^{|\mathcal{D}|}\end{array}$$

$\mathcal{D}$: index set of discrete variables.

$\mathbf{x} = [\mathbf{x}_{\mathcal{D}}, \mathbf{x}_{\mathcal{C}}]$: discrete + continuous.

Key Challenge

The integrality constraint makes the feasible set *non-convex* and *combinatorially large*. NP-hard in general.

Totally Unimodular Matrices

Definition

A matrix **A** is **totally unimodular (TU)** if every square submatrix has determinant $\in \{-1, 0, 1\}$.

- If **A** is TU and $\mathbf{b} \in \mathbb{Z}^m$, then all LP optimal vertices are **automatically integral**
- The LP relaxation solves the IP exactly!
- Example: network flow constraint matrices are TU
- Single-entry (0/1) matrices with at most one 1 per column that appear in bi-partite graphs are TU

Totally Unimodular (TU) Matrices

Non-TU: det = -1 or 2 submatrix		TU: all subdeterminants $\in \{-1, 0, 1\}$										
<table><tr><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>		1	1	1	0	<table><tr><td>1</td><td>-1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>-1</td></tr></table>	1	-1	0	0	1	-1
1	1											
1	0											
1	-1	0										
0	1	-1										

LP Relaxation and Rounding

LP Relaxation

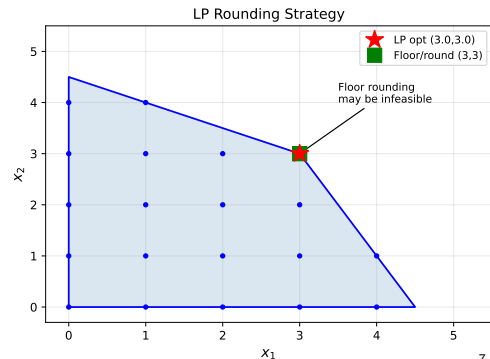
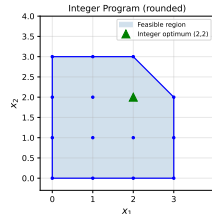
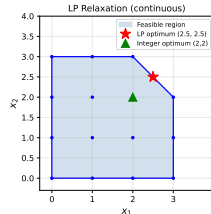
Drop the integrality constraint $\mathbf{x} \in \mathbb{Z}^n$ and solve the resulting LP. This gives a **lower bound** on the IP optimum.

Rounding strategies:

- 1 **Simple floor/ceiling:** $\hat{x}_i = \lfloor x_i^* \rfloor$ or $\lceil x_i^* \rceil$
Problem: may be infeasible!
- 2 **Probabilistic rounding:** round x_i^* up with probability equal to its fractional part
- 3 **Iterative rounding:** fix most-rounded component, re-solve

Caveat

For some problems, rounding yields a solution that is far from optimal or *infeasible*.



Rounding — Python Example

```
1 import numpy as np
2 from scipy.optimize import linprog
3
4 # Integer program: minimize c @ x
5 # subject to A_ub @ x <= b_ub
6 # x >= 0, x in Z^n
7 c = np.array([1.0, 2.0]) # objective
8 A_ub = np.array([[ -1.0, -1.0], # -x1 - x2 <= -3 (i.e. x1+x2>=3)
9                  [ 1.0, 2.0]]) # x1 + 2x2 <= 7
10 b_ub = np.array([-3.0, 7.0])
11 bounds = [(0, None), (0, None)]
12
13 # Step 1: solve LP relaxation
14 res = linprog(c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method='highs')
15 x_lp = res.x
16 print(f"LP relaxation: x = {x_lp}, obj = {res.fun:.4f}")
17
18 # Step 2: simple rounding (floor)
19 x_floor = np.floor(x_lp).astype(int)
20 x_ceil = np.ceil(x_lp).astype(int)
21 print(f"Floor rounded: {x_floor}, obj = {c @ x_floor:.4f}")
22 print(f"Ceil rounded: {x_ceil}, obj = {c @ x_ceil:.4f}")
23
24 # Step 3: check feasibility of rounded solutions
25 for name, xr in [('floor', x_floor), ('ceil', x_ceil)]:
26     feasible = np.all(A_ub @ xr <= b_ub + 1e-8)
27     print(f"{name}: feasible = {feasible}")
```

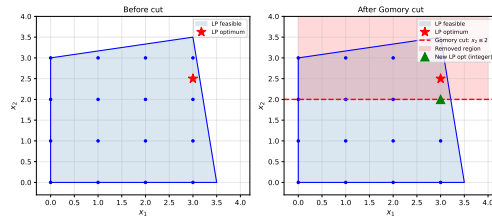

Cutting Plane Method

Idea: iteratively add *valid cuts* (linear inequalities) that eliminate the current fractional LP optimum while not removing any integer-feasible point.

Procedure:

- 1 Solve the LP relaxation; get solution $\mathbf{x}^*$
- 2 If $\mathbf{x}^*$ is integral, we are done
- 3 Find a cutting plane that: (a) cuts off $\mathbf{x}^*$, and (b) is satisfied by all integer-feasible points
- 4 Add the cut to the LP, go to step 1

Each iteration **tightens** the LP relaxation, moving the LP optimum toward an integer solution.



Left: LP optimum is fractional. Right: after adding a Gomory cut, the LP optimum is integer-valued.

Gomory Cuts — Derivation

Gomory Cut (Eq. 22.10–22.11)

Given a non-integral LP optimal solution $\mathbf{x}^*$ (basis B), for a non-integral basic variable x_i with $\bar{x}_i \notin \mathbb{Z}$:

$$x_i + \sum_{j \notin B} (\lfloor \bar{A}_{ij} \rfloor - \bar{A}_{ij}) x_j \leq \lfloor \bar{b}_i \rfloor \quad (22.11)$$

where $\bar{\mathbf{A}} = \mathbf{A}_B^{-1} \mathbf{A}_V$ and $\bar{\mathbf{b}} = \mathbf{A}_B^{-1} \mathbf{b}$.

Symbols:

- B : set of basic variable indices; $\mathbf{A}_B$: basic submatrix
- V : set of non-basic (variable) indices
- $\bar{A}_{ij}$: (i, j) entry of the reduced basis matrix
- $\lfloor \cdot \rfloor$: floor function; $\text{frac}(x) = x - \lfloor x \rfloor$

Key Property

The cut is *valid*: all integer-feasible points satisfy it. It is *tight*: the current LP solution violates it.

Gomory Cut — Running Example (Ex. 22.3)

Problem

$$\underset{\mathbf{x}}{\text{minimize}} \quad 2x_1 + x_2 + 3x_3 \quad \text{s.t.} \quad \begin{bmatrix} 0.5 & -0.5 & 1.0 \\ 2.0 & 0.5 & -1.5 \end{bmatrix} \mathbf{x} = \begin{bmatrix} 2.5 \\ -1.5 \end{bmatrix}, \quad \mathbf{x} \geq \mathbf{0}, \quad \mathbf{x} \in \mathbb{Z}^3$$

Iteration 1: LP relaxation gives $\mathbf{x}^* \approx [0.818, 0, 2.091]$ (non-integral). Gomory cuts derived:

$$x_4 - 0.909x_2 = -0.818 \quad x_5 - 0.545x_2 = -0.091$$

Iteration 2: Solve augmented LP $\Rightarrow \mathbf{x}^* \approx [0.9, 0.9, 2.5, 0.0, 0.4]$. Still fractional. Add more cuts.

Iteration 3: Solve $\Rightarrow \mathbf{x}^* = [1, 2, 3, 1, 1, 0, 0, 0, 0]$. **Final solution:** $\mathbf{x}_i^* = [1, 2, 3]$.

Complexity: Each iteration adds one constraint; convergence is finite for bounded IPs (Gomory, 1958), but may need exponentially many cuts.

Cutting Plane Algorithm — Python

```
1 import numpy as np
2 from scipy.optimize import linprog
3
4 def frac(x):
5     return x - np.floor(x)
6
7 def cutting_plane_ip(c, A_eq, b_eq, n_orig, max_iter=30):
8     """Gomory cutting plane for equality-form IP ( $x \geq 0$ ,  $x$  in  $\mathbb{Z}^{n\_orig}$ )."""
9     A, b = A_eq.copy(), b_eq.copy()
10    c_aug = np.concatenate([c, np.zeros(0)])
11
12    for iteration in range(max_iter):
13        # Solve LP relaxation
14        res = linprog(c_aug, A_eq=A, b_eq=b,
15                     bounds=[(0, None)]*A.shape[1], method='highs')
16        if res.status != 0:
17            raise ValueError("LP infeasible or unbounded")
18        x = res.x
19
20        # Check integrality of original variables
21        frac_vals = frac(x[:n_orig])
22        if np.all(frac_vals < 1e-6):
23            return np.round(x[:n_orig]).astype(int)
24
25        # Find most fractional variable
26        i = np.argmax(frac_vals)
27        fi = frac_vals[i]
```

Branch and Bound — Overview

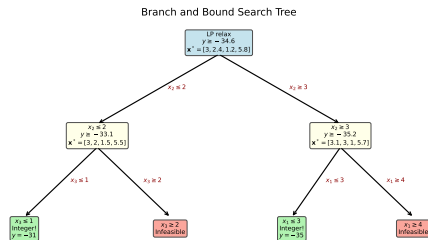
Idea

Enumerate solutions implicitly by *branching* on a fractional variable and maintaining *bounds* to prune the search tree.

Three operations:

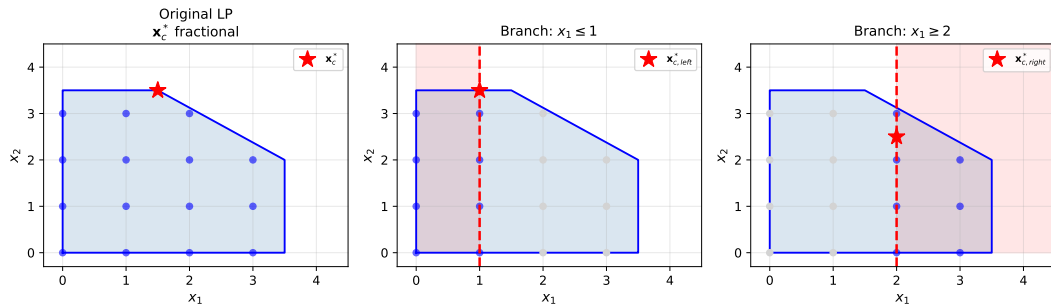
- 1 **Branch:** split on a non-integral component x_i^* into two sub-LPs: $x_i \leq \lfloor x_i^* \rfloor$ and $x_i \geq \lceil x_i^* \rceil$
- 2 **Bound:** solve LP relaxation of each child to get a lower bound $y \geq \mathbf{c}^T \mathbf{x}^*$
- 3 **Prune:** discard branches where the lower bound exceeds the current best integer solution (*incumbent*)

Priority: use a priority queue ordered by LP bound; explore the most promising node first.



Branching Splits the Feasible Set (Fig. 22.4)

Branching Splits the Feasible Set



When $x_1^* = 1.5$ (fractional), we split into two sub-problems: $x_1 \leq 1$ (left) and $x_1 \geq 2$ (right). Each sub-problem has fewer feasible integer points.

Branch and Bound — Branching Example (Ex. 22.4)

Setup

LP relaxation gives $\mathbf{x}_c^* = [3, 2.4, 1.2, 5.8]$ with $\mathbf{c} = [-1, -2, -3, -4]$. Lower bound: $y \geq \mathbf{c}^\top \mathbf{x}_c^* = -34.6$.

Branching step:

- Choose the non-integral component farthest from an integer: $x_{2,c}^* = 2.4$ (distance 0.4 from 2 or 3)
- Create two sub-problems:
 - **Left:** add $x_2 \leq 2$
 - **Right:** add $x_2 \geq 3$
- Solve each sub-LP to get new lower bounds
- Keep expanding the most promising leaf

Pruning Conditions

A node is pruned if: (a) LP is infeasible, (b) LP bound $\geq$ current best integer objective, (c) LP solution is already integral.

Branch and Bound — Python Implementation

```
1 import numpy as np
2 from scipy.optimize import linprog
3 import heapq
4
5 def branch_and_bound(c, A_ub, b_ub, int_vars, bounds=None):
6     """Branch and bound for minimization IP.
7         int_vars: list of indices that must be integer."""
8     n = len(c)
9     if bounds is None:
10         bounds = [(0, None)] * n
11     best_val = np.inf
12     best_x = None
13     # Priority queue: (LP_bound, id, A_ub, b_ub, bounds)
14     counter = [0]
15     def push(A, b, bds):
16         res = linprog(c, A_ub=A, b_ub=b, bounds=bds, method='highs')
17         if res.status == 0:
18             heapq.heappush(pq, (res.fun, counter[0], A, b, bds, res.x))
19             counter[0] += 1
20     pq = []
21     push(A_ub, b_ub, bounds)
22     while pq:
23         lb, _, A, b, bds, x = heapq.heappop(pq)
24         if lb >= best_val:           # prune: bound exceeds incumbent
25             continue
26         frac = {i: x[i] - np.floor(x[i]) for i in int_vars}
27         if abs(x[i] - round(x[i])) > 1e-6}
```


Branch and Bound — Complexity and Notes

Complexity

- Worst case: $O(2^n)$ nodes (full binary tree)
- In practice: effective pruning makes B&B much faster
- LP solve at each node: $O(m^2n)$ (simplex)
- Total typical: polynomial-ish on structured problems

Improvements in practice:

- **Variable selection:** choose most fractional or using pseudo-costs
- **Node selection:** best-first, depth-first, or hybrid
- **Warm starts:** reuse LP basis from parent node
- **Combine** with cutting planes (*branch-and-cut*, used in Gurobi/CPLEX)

Example: IP with $n = 4$

$$\mathbf{c} = [-1, -2, -3, -4]$$

$$\mathbf{x}_c^* = [3, 2.4, 1.2, 5.8]$$

$$y \geq -34.6 \text{ (LP bound)}$$

Branch on x_2 (farthest from integer).

After 3 levels, integer solution found.

Dynamic Programming for Discrete Optimization

Key Idea

Exploit **optimal substructure**: the optimal solution to a large problem can be composed from optimal solutions to smaller *overlapping subproblems*.

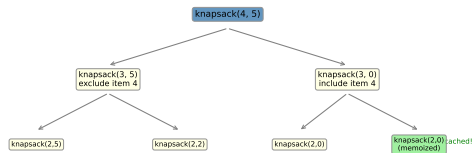
Two implementations:

- 1 **Top-down (memoization)**: recurse with caching
- 2 **Bottom-up (tabulation)**: fill table iteratively

When applicable:

- Problem has *optimal substructure*
- Subproblems *overlap* (same state visited many times)
- State space is *manageable* in size

DP Recursion Tree with Memoization
(overlapping subproblems are computed once)



Memoized subproblems (green) are computed once and cached.

The 0-1 Knapsack Problem

Problem Formulation (Eq. 22.13)

Given n items with weights w_i and values v_i , and capacity $w_{\max}$:

$$\underset{\mathbf{x}}{\text{minimize}} \quad - \sum_{i=1}^n v_i x_i \quad \text{s.t.} \quad \sum_{i=1}^n w_i x_i \leq w_{\max}, \quad x_i \in \{0, 1\}$$

2^n possible solutions — direct enumeration intractable.

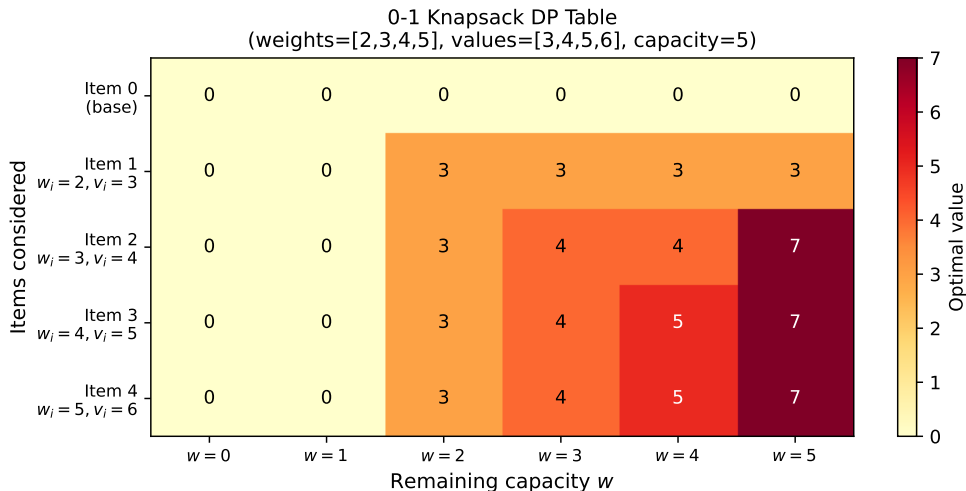
DP Recurrence (Eq. 22.14)

Define $\text{knapsack}(i, w_{\max}) = \text{max value using items } 1, \dots, i \text{ with capacity } w_{\max}$:

$$\text{knapsack}(i, w_{\max}) = \begin{cases} 0 & i = 0 \\ \text{knapsack}(i-1, w_{\max}) & w_i > w_{\max} \\ \max \begin{cases} \text{knapsack}(i-1, w_{\max}) & \text{(discard)} \\ \text{knapsack}(i-1, w_{\max} - w_i) + v_i & \text{(include)} \end{cases} & \text{otherwise} \end{cases}$$

Complexity: $O(n \cdot w_{\max})$ time and space (pseudo-polynomial)

Knapsack DP Table Visualization



Items: weights = [2, 3, 4, 5], values = [3, 4, 5, 6], capacity = 5. Each cell shows the maximum value achievable using the first i items with remaining capacity w . Optimal value = 7 (items 1 and 2).

Knapsack DP — Python Implementation

```
1 import numpy as np
2
3 def knapsack(weights, values, w_max):
4     """0-1 Knapsack via bottom-up DP.
5     Returns (max_value, selected_items_mask)."""
6     n = len(weights)
7     # dp[i][w] = max value with items 0..i-1 and capacity w
8     dp = np.zeros((n + 1, w_max + 1), dtype=float)
9
10    for i in range(1, n + 1):
11        wi, vi = weights[i-1], values[i-1]
12        for w in range(w_max + 1):
13            dp[i, w] = dp[i-1, w]           # exclude item i
14            if wi <= w:
15                dp[i, w] = max(dp[i, w],
16                               dp[i-1, w - wi] + vi) # include item i
17
18    # Traceback to find selected items
19    selected = np.zeros(n, dtype=bool)
20    w = w_max
21    for i in range(n, 0, -1):
22        if dp[i, w] != dp[i-1, w]:
23            selected[i-1] = True
24            w -= weights[i-1]
25
26    return dp[n, w_max], selected
27
```

DP — Top-Down (Memoized) Implementation

```
1 from functools import lru_cache
2
3 def knapsack_topdown(weights, values, w_max):
4     """Top-down memoized knapsack."""
5     n = len(weights)
6
7     @lru_cache(maxsize=None)
8     def dp(i, w):
9         """Max value using items i..n-1 with remaining capacity w."""
10        if i == n:
11            return 0.0
12        # Option 1: skip item i
13        result = dp(i + 1, w)
14        # Option 2: include item i (if it fits)
15        if weights[i] <= w:
16            result = max(result, values[i] + dp(i + 1, w - weights[i]))
17        return result
18
19    return dp(0, w_max)
20
21 # Example
22 weights = [2, 3, 4, 5]
23 values = [3, 4, 5, 6]
24 print(knapsack_topdown(weights, values, w_max=5)) # 7
```

Listing 5: Top-down memoized knapsack (Python)

Ant Colony Optimization — Motivation

Biological Inspiration

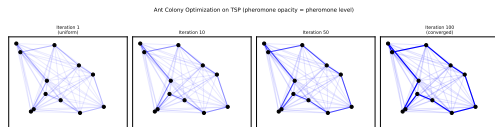
Real ants deposit *pheromones* on paths; shorter paths accumulate more pheromone over time, attracting more ants — positive feedback reinforces good routes.

ACO applications:

- Traveling Salesman Problem (TSP)
- Vehicle routing
- Network routing (IP packet routing)
- Job scheduling

Key components:

- Pheromone matrix $\tau(i \rightarrow j)$
- Heuristic (edge attractiveness) $\eta(i \rightarrow j)$
- Transition probability based on both
- Pheromone evaporation ρ



ACO on TSP: pheromone level shown by edge opacity. Path quality improves with iterations.

ACO — Edge Attractiveness and Transition Probability

Edge Attractiveness (Eq. 22.15)

The attractiveness of edge $(i \rightarrow j)$ combines pheromone and prior:

$$A(i \rightarrow j) = \tau(i \rightarrow j)^\alpha \cdot \eta(i \rightarrow j)^\beta$$

where:

- $\tau(i \rightarrow j)$: pheromone level on edge $(i \rightarrow j)$
- $\eta(i \rightarrow j) = 1/\text{length}(i \rightarrow j)$: heuristic (inverse length)
- α : pheromone exponent (default 1.0)
- β : prior exponent (default 5.0)

Transition Probability (Eq. 22.16)

From node i with unvisited neighbors $\mathcal{N}$:

$$P(i \rightarrow j) = \frac{A(i \rightarrow j)}{\sum_{k \in \mathcal{N}} A(i \rightarrow k)} \quad \forall j \in \mathcal{N}$$

ACO — Pheromone Update and Evaporation

Pheromone Update (Eq. 22.17)

After all m ants complete their tours at iteration k :

$$\tau(i \rightarrow j)^{(k+1)} = (1 - \rho) \tau(i \rightarrow j)^{(k)} + \sum_{a=1}^m \frac{1}{\ell^{(a)}} \cdot \mathbf{1}[(i \rightarrow j) \in \mathcal{P}^{(a)}]$$

where:

- $\rho \in [0, 1]$: evaporation rate (common: $\rho = 0.5$)
- $\ell^{(a)}$: total path length of ant a 's tour
- $\mathcal{P}^{(a)}$: set of edges traversed by ant a
- $(1 - \rho)$: evaporation factor — prevents premature convergence

Evaporation

Without evaporation, early bad paths accumulate pheromone and the algorithm gets stuck. $\rho = 1/2$ is a common default.

ACO — Python Implementation

```
1 import numpy as np
2
3 def ant_colony_optimization(coords, n_ants=50, n_iter=100,
4                             alpha=1.0, beta=5.0, rho=0.5):
5     """ACO for TSP. coords: (n,2) array of city positions."""
6     n = len(coords)
7     # Distance matrix
8     dist = np.linalg.norm(coords[:, None] - coords[None, :], axis=2)
9     dist[dist == 0] = 1e-10
10    eta = 1.0 / dist                # heuristic
11    tau = np.ones((n, n))          # pheromone (init uniform)
12
13    best_tour, best_len = None, np.inf
14
15    for _ in range(n_iter):
16        tours, lengths = [], []
17        for ant in range(n_ants):
18            visited = [0]
19            for step in range(n - 1):
20                curr = visited[-1]
21                unvisited = [j for j in range(n) if j not in visited]
22                # Compute transition probabilities
23                attract = np.array([tau[curr,j]**alpha * eta[curr,j]**beta
24                                   for j in unvisited])
25                probs = attract / attract.sum()
26                nxt = unvisited[np.random.choice(len(unvisited), p=probs)]
27                visited.append(nxt)
```

ACO Algorithm Summary

Algorithm (Alg. 22.10):

- ① Initialize $\tau(i \rightarrow j) = 1$ for all edges
- ② For $k = 1, \dots, k_{\max}$:
 - ① Compute edge attractiveness $A(i \rightarrow j)$
 - ② Apply evaporation: $\tau \leftarrow (1 - \rho)\tau$
 - ③ For each of m ants: simulate tour using transition probabilities (Alg. 22.9)
 - ④ Update pheromones from successful tours
- ③ Return best tour found

Parameters: $m = 1000$, $k_{\max} = 100$, $\alpha = 1.0$, $\beta = 5.0$,
 $\rho = 0.5$

Convergence: Not guaranteed to global optimum; a meta-heuristic with good empirical performance.

ACO vs. Other Methods

Method	Exact?	Scalable?
Cutting plane	Yes	Moderate
Branch & bound	Yes	Moderate
Dynamic prog.	Yes	Limited
ACO	No	Yes

Summary

- **Integer programs** are LPs with integrality constraints; the feasible set is discrete and non-convex.
- **LP relaxation** gives a continuous lower bound; rounding may be infeasible or suboptimal, but is central to advanced methods.
- **Cutting planes (Gomory cuts)** iteratively tighten the LP relaxation by adding valid linear cuts, convergent in finite iterations.
- **Branch and bound** enumerates candidates implicitly using LP bounds; combined with cutting planes it powers modern MIP solvers (Gurobi, CPLEX).
- **Dynamic programming** exploits optimal substructure and overlapping subproblems; pseudo-polynomial for knapsack ($O(nW)$).
- **Ant colony optimization** is a nature-inspired meta-heuristic for path-based problems; pheromone reinforces good solutions while evaporation prevents premature convergence.

Selected Exercises I

Exercise 22.1 (Boolean Satisfiability)

Find $\mathbf{x}$ satisfying $f(\mathbf{x}) = x_1 \wedge (x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2)$ by enumeration. *Solution:* $(x_1, x_2, x_3) = (\text{false}, \text{true}, \text{true})$. Worst-case enumeration: 2^n for n -dim vector.

Exercise 22.2 (SAT as IP)

Convert the Boolean expression to an integer linear program. Use $x_i \in \{0, 1\}$ ($1 = \text{true}$, $0 = \text{false}$); negate via $1 - x_i$.

$$x_1 \geq 1, \quad x_2 - x_3 \geq 0, \quad -x_1 - x_2 \geq -1, \quad \mathbf{x} \leq \mathbf{1}, \quad \mathbf{x} \in \mathbb{N}^3$$

Any Boolean satisfiability problem can be cast as an IP.

Exercise 22.3 (Totally Unimodular)

TU matrices are important because when $\mathbf{A}$ is TU and $\mathbf{b}$ is integral, all LP optimal vertices are integral, so the LP relaxation solves the IP exactly. A TU matrix can only have entries $\in \{-1, 0, 1\}$ because the determinant of any 1×1 submatrix must be in $\{-1, 0, 1\}$.

Exercise 22.5 (Big-M Method)

Selected Exercises II

Binary variable z selects one of two constraints. Using large M :

$$\mathbf{a}_0^\top \mathbf{x} \leq b_0 + Mz, \quad \mathbf{a}_1^\top \mathbf{x} \leq b_1 + M(1 - z), \quad z \in \{0, 1\}$$

When $z = 0$: first constraint active; when $z = 1$: second active. The “big M method” keeps the problem an MILP.

Exercise 22.4 (Knapsack Branch & Bound)

Items $\{1, \dots, 6\}$, weights $= [9, 2, 3, 4, 8, 6]$, values $= [9, 3, 5, 3, 4, 6]$, $w_{\max} = 20$. Branch on x_1 :

- $x_1 = 0$: sort remaining by value/weight; lower bound $= -12$
- $x_1 = 1$: lower bound ≈ -18.333

Final solution: $\mathbf{x} = [1, 0, 0, 0, 1, 1]$.

Method Comparison

Method	Exact?	Complexity	IP type	Scalability	Key idea
LP Rounding	Approx	$O(\text{LP})$	General	High	Relax then round
Cutting planes	Yes	Exp. worst	IP/MIP	Medium	Add Gomory cuts
Branch & bound	Yes	$O(2^n)$ worst	IP/MIP	Medium	Tree enumeration
Branch & cut	Yes	Better avg	IP/MIP	High	B&B + cuts
Dynamic prog.	Yes	$O(n \cdot W)$	Special	Low–Med	Tabulation
Ant colony opt.	Heuristic	$O(mk_{\max} n^2)$	Path/graph	High	Pheromone

- Modern MIP solvers (Gurobi, CPLEX) combine branch-and-cut with advanced heuristics to solve problems with millions of variables
- DP is exact but limited to problems with polynomial state spaces
- ACO and other metaheuristics trade optimality for scalability

References

- Kochenderfer & Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026. Chapter 22.
- R. E. Gomory, "Outline of an algorithm for integer solutions to linear programs," *Bull. Amer. Math. Soc.*, 64(5):275–278, 1958.
- M. Dorigo, V. Maniezzo, A. Coloni, "Ant system: optimization by a colony of cooperating agents," *IEEE Trans. SMC*, 26(1):29–41, 1996.
- L. A. Wolsey, *Integer Programming*, Wiley, 1998.
- G. L. Nemhauser & L. A. Wolsey, *Integer and Combinatorial Optimization*, Wiley, 1988.